buildings

MDPI

Article

A Semantic Web and IFC-Based Framework for Automated BIM Compliance Checking

Lu Jia [1], Maokang Chen [1], Chen Chen [2,*] and Yanfeng Jin [1]

[1] School of Infrastructure Engineering, Nanchang University, 999# Xuefu Ave., Honggutan District, Nanchang 330031, China; jialuuser@163.com (L.J.); c15216088276@163.com (M.C.); jinyanfeng@email.ncu.edu.cn (Y.J.)
[2] School of Architecture, Zhengzhou University, No. 100 Science Avenue, High Tech Zone, Zhengzhou 450001, China
* Correspondence: chenuser0913@163.com; Tel.: +86-189-7088-7890

Abstract

In the architectural design phase, the inspection of design deliverables is critical, yet traditional manual checking methods are time-consuming, labor-intensive, and inefficient, with numerous drawbacks. With the development of BIM technology, automated rule compliance checking has become a trend. This paper presents a method combining semantic web technology and IFC data to enhance human–machine collaborative inspection capabilities. First, a five-step process integrated with domain specifications is designed to construct a building object ontology, covering most architectural objects in the AEC domain. Second, a set of mapping rules is developed based on the expression mechanisms of IFC entities to establish a semantic bridge between IfcOWL and the building object ontology. Then, by analyzing regulatory codes, query rule templates for major constraint types are developed using semantic web SPARQL. Finally, the feasibility of the method is validated through a case study based on the Jena framework.

Keywords: building information modeling; semantic web; ontology; compliance checking; natural language processing

Received: 19 June 2025
Revised: 14 July 2025
Accepted: 24 July 2025
Published: 25 July 2025

Citation: Jia, L.; Chen, M.; Chen, C.; Jin, Y. A Semantic Web and IFC-Based Framework for Automated BIM Compliance Checking. *Buildings* **2025**, *15*, 2633. https://doi.org/10.3390/buildings15152633

1. Introduction

During the architectural design phase, the inspection of design deliverables represents a critical task, as the quality of such deliverables exerts significant impacts on the construction efficiency and building quality during the construction phase, as well as the utilization benefits during the operation and maintenance phase. The traditional inspection method relies on industry standards and work experience to conduct manual inspections on 2D drawings. Such inspection method is time-consuming, labor-intensive, error-prone, and inefficient.

With the emergence of BIM technology, the dimension of inspection has risen from 2D to 3D, and the depth and ways of information expression have been greatly improved. The visual expression makes it convenient for inspectors to observe contents that could not be expressed in previous 2D drawings. Moreover, as an integrated information database, it can be combined with computer technology to develop intelligent auxiliary inspection methods. Therefore, automatic compliance checking has become a research focus in the AEC field. Nowadays, automated compliance checking methods have been studied and applied in many fields such as building energy efficiency [1], building structural design [2], construction technology [3], constructability [4], and fire safety [5].

Early automated compliance checks mainly relied on hard-coded approaches. For example, Foo Sing, T et al. [6] used the IFC model as the database to develop a Construction and Real Estate Network (CORENET) platform based on C++ coding. Jotne EPM Technology released the EXPRESS data management platform designed to convert specifications into the EXPRESS modeling language. The Solibri Model Checker (SMC) platform systematically reads IFC models, standard data schemas, and allows users to adjust the parameters of the rules. However, this method requires high programming skills from professionals in the field [7], and a lot of maintenance work is needed when data is updated [8], making it hard for professionals in the field to reuse it. Therefore, more research has focused on translating regulations using logical statements and ontology markup languages.

Eastman et al. [9] first summarized the rule checking process into four phases: (1) Rule Expression Phase: Translating regulations or standards expressed in natural language form into the form of computer-readable language; (2) Model Data Preparation Phase: Extracting the model data that needs to be checked from the BIM model; (3) Rule Enforcement Phase: Executing computer-processable rules to check the extracted model data; (4) Check result reporting phase: Presenting the results of compliance checking to users.

Based on this process, research on automated compliance checking has made many advancements. For example, Pauwels et al. [10] proposed a method for evaluating and checking the acoustic performance of buildings. This method converts the content of relevant standards into a computer-recognizable format using RDF technology. Hjelseth, E. et al. [11] proposed the RASE semantic mark-up method, which is used capture and structure the normative constraints in building specifications, and realize computer interpretability, thereby supporting automated code checking and compliance verification in the BIM environment. Goh et al. [12] proposed a framework for an automated compliance checking system in a BIM environment, covering aspects such as design specification classification, model preparation, rule conversion, and system design.

Although existing research has made significant progress in automated compliance checking, these technologies still face challenges in terms of scalability and efficiency. Therefore, this paper proposes a method that combines semantic web technology with IFC, the exchange standard for BIM. First, we designed a five-step method for ontology development. By collecting classification standards for building objects in the field, a reusable building object ontology is developed. Secondly, the model data were represented in the exchangeable data format IFC and converted to IfcOWL ontology by a conversion tool. Then, after fully analyzing the expression and relationships of entity information under the IFC system, mapping rules are designed based on the Jena language and its ARQ function library to address the heterogeneity between the building object ontology and the IfcOWL ontology, which are ultimately merged into a merged ontology. Finally, we analyzed and classified the expression structures of different specification statements and used SPARQL statements to design templates for translating different types of specifications. Then, we conducted automated compliance checking based on the Jena framework.

2. Related Work

2.1. Code Ontology

The concept of ontology originated in philosophy, where it describes the abstract essence of objective existence and explores the nature and laws governing entities [13]. One of the most appropriate definitions of an ontology is "the basic terms and relationships that make up a vocabulary of subject matter domains, and the rules used to combine the terms and relationships to define the extents of the vocabulary" [14]. In general, ontology is part of the semantic web stack and is used to facilitate communication, sharing, annotation of information, and reuse of domain knowledge [15]. Consequently, ontology

technologies (alongside the semantic web and linked data) have gained significant traction and demonstrated success in diverse fields such as biology, healthcare, accounting, and social media [16]. These successful cases have promoted the application of ontologies in the AEC area [17].

Demir [18] pointed out that most codes are presented in text form. In order to provide more computerized support for users of codes, it is necessary to represent code knowledge in a formal and computer-interpretable way. Ontologies allow for reasoning and are capable of inferring new knowledge to assess the compliance of codes [19]. It is feasible to computerize the codes within a domain using the language of ontologies. For example, Zhong et al. [20] designed a quality inspection and evaluation ontology for the construction quality of a project to conduct compliance inspections of building quality. Lu et al. [21] proposed a Construction Safety Check Ontology (CSCOontology) based on construction safety standards and divided its concepts into five categories: "Line of work", "Task", "Precursor", "Hazard", and "Solution". Ma et al. [22] computerized the BIM quality standards using OWL to form a BIM-QC ontology for checking the quality of BIM models. Fitkau et al. [23] proposed a Fire Safety Ontology (FiSa), which can classify buildings and their components according to fire safety considerations and helps determine the compliance of building designs based on the regulations of all relevant fields in the design. The core of these studies is to convert the codes represented in natural language into ontology language to achieve the goal of computer readability. However, the code ontologies in most studies are limited to a specific code. When facing different inspection requirements, new ontologies need to be redeveloped, reducing the reusability of the ontology and lowering the inspection efficiency.

2.2. Linking BIM Data

Linked data is a technology originating from semantic web research and based on W3C standards. It uses open web protocols to share structured data, has the characteristics of openness, modularity, and scalability, follows specific principles, and can achieve cross—domain data sharing, enhance interoperability, and transform the way data is used [24]. In the AEC field, the IfcOWL ontology is the most widely used linked data. It is derived from the conversion of IFC data represented in EXPRESS and described in OWL language, which can be used for various tasks such as design [25], construction [26], operation [27], and rule checking [28]. Currently, IfcOWL has become the standard of BuildingSMARTInternational [29]. The key to the conversion from IFC to IfcOWL lies in the mapping between the two. Schevers and Drogemuller [30] achieved the first conversion of IFC to IfcOWL. Pauwels et al. [31] developed an EXPRESS-To-OWL converter, which enables more convenient conversion of IFC data.

During the process of performing automated compliance checks, heterogeneity in ontology descriptions may occur between the code ontology and IfcOWL. For example, spatial entities such as "kitchen" and "bedroom" in the code ontology are classified as IfcSpace in IfcOWL. In such cases, it is necessary to rely on other attributes, such as the attribute Function Type in IfcProperty, along with values like "kitchen" and "bedroom" for differentiation. Therefore, it is essential to design mapping rules between the code ontology and IfcOWL.

2.3. Rule Language

One of the most challenging tasks in the process of automated compliance checking is the interpretation of rules. Currently, a commonly used method is to translate specifications into computer-executable rules using rule languages. For example, Zhang et al. [32] proposed an optimization strategy using SPARQL query rules for model checking to ensure

quality. Du et al. [33] used SWRL rules to convert standard-based defect assessment methods into rules for rating defect levels such as cracks and leaks. In addition, some specialized semantic web rules, such as SWQRL [34], SHACL [35], and N3Logic language [36], are also widely applied for automated compliance checking. These rule languages are easy to understand, and domain professionals can encode with them effortlessly, such as the Building Environment Regulatory Language (BERA) [37], KBim [38], BIM Rule Language (BIMRL) [39], and Doors Rule Language [5].

2.4. Summary of Related Work

In conclusion, Table 1 summarizes some key recent studies on automated compliance checking. Based on a summary of existing research, the conclusions drawn are as follows. (1) The code ontologies developed in some studies are limited to specific standards. When faced with different requirements for specification inspection, the reusability of existing ontologies may be insufficient, making it necessary to redevelop new ontologies. This increases the workload of inspection and thus reduces inspection efficiency. (2) Most existing studies use SWRL for rule expression. However, when performing ontology mapping, the mapping or reasoning process facing complex spatial information (including large-scale triples) will be slightly strenuous, and it is often difficult to achieve the desired results.

Table 1. Key studies related to automated compliance checking.

Study	Method Description	Limitations
Eastman et al. [9]	Proposed a rule-based method to implement compliance checks in architectural design.	Difficulty in dealing effectively with complex or ambiguous building code requirements.
Pauwels et al. [10]	Using semantic web technology (RDF + rule languages) for IFC model checking.	High complexity of rule writing; cross-domain rule integration mechanisms are immature.
Hjelseth, E. et al. [11]	Proposed the RASE semantic markup method to capture knowledge in building codes.	Rely on manual marking standards; the issue of BIM model data missing remains unresolved.
Goh et al. [12]	Developed an automated compliance checking system in a BIM-based environment.	Limited scope of application; manual-dependent model entry; long rule execution time.
Zhong et al. [20]	Construction quality compliance inspection based on ontology and semantic rules.	Cannot directly handle regulatory constraints involving spatial relationships.
Lu et al. [21]	Automated safety inspection and risk reasoning based on ontology and semantic rules.	JESS has limited reasoning ability and insufficient support for complex logic.

3. Methodology

The essence of automatic compliance checking is to query whether there are building objects that do not meet the requirements of the codes in the building model. A complete code statement can be summarized as consisting of three parts: constrained objects, constrained contents, and preconditions. To achieve automatic compliance checking, the key lies in how to computerize the inspection basis and the inspection objects. In this paper, we adopt the combination of ontology technology and the BIM model for automatic compliance checking. The summary of the inspection idea is shown in Figure 1 below.

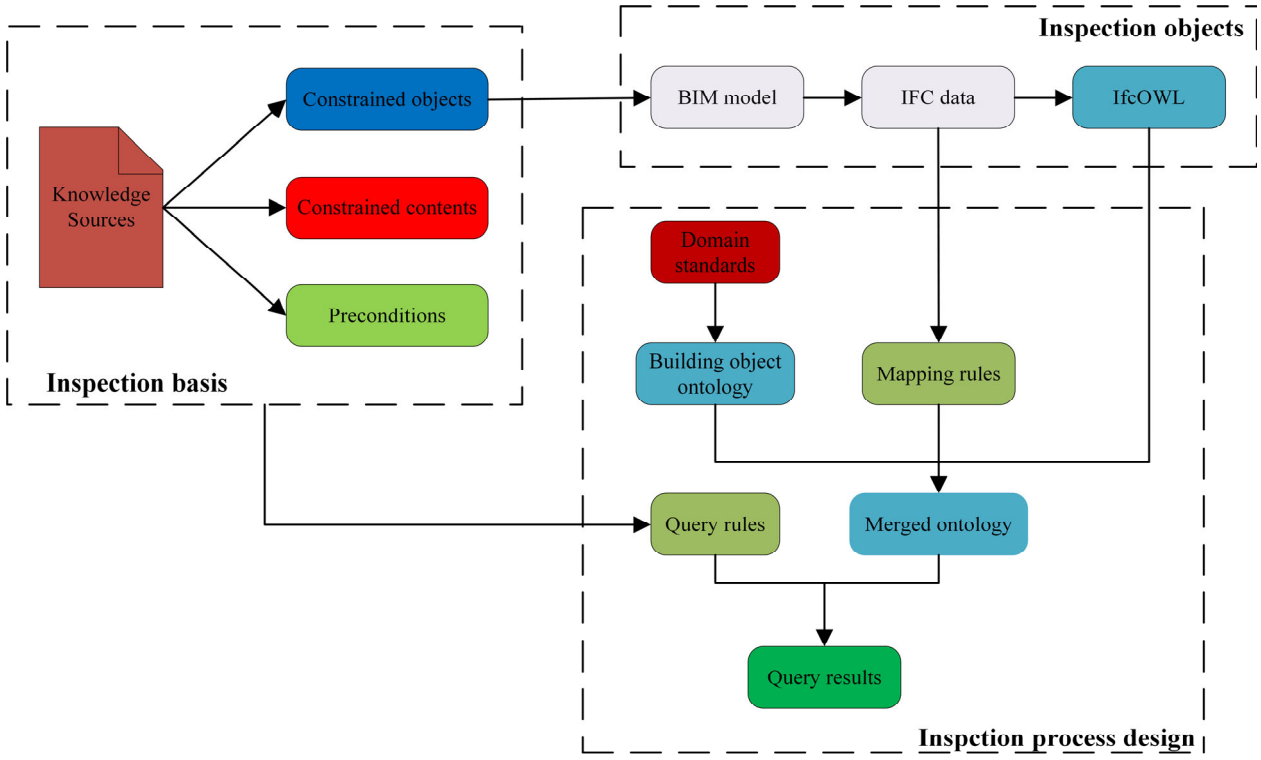

Figure 1. Framework structure of the inspection.

In the inspection basis, the constrained objects in the knowledge sources correspond to the building objects in the BIM model data subject to be inspected. The constrained contents serve as the basis for determining whether the building objects meet the requirements of the codes, and the preconditions are used to filter out individual cases of building objects that need to meet the constraints.

The inspection objects are sourced from the BIM model, and its IFC format files, serving as the direct data source for carrying the information of the building objects within it. Currently, it is the most powerful and comprehensive open data exchange standard, with a complete building information description system.

The inspection process design consists of two phases: an ontology development phase and a rule development phase. The ontology development phase includes two ontologies: the building object ontology and the IfcOWL ontology. Among them, the building object ontology is developed based on the domain standards, and the IfcOWL ontology is converted from IFC files. The rule development phase consists of two sets of rules: mapping rules and query rules. Specifically, the mapping rules are expressed in Jena language and developed according to the expressions of building objects and related information in the IFC standard. These mapping rules can resolve the heterogeneity between the building object ontology and the IfcOWL ontology and integrate them into a merged ontology. The query rules are expressed in SPARQL statements, based on specific codes to formulate query rules.

4. Ontology Development

Previously, academic organizations and research institutes have proposed different ontology construction methods for their own needs. Commonly used methods for creating ontologies include the IDEF5 Method [40], the TOVE Method [41], the Methontology method [42], and the Seven-step Iterative Method [43]. Among them, the seven-step method is the most widely used, as it has a more detailed description and a more systematic framework than

the other methods. Its basic steps include defining domain and scope; considering reusing existing ontologies; enumerating important terms; defining classes and hierarchy; defining properties of classes; defining constraints and axioms; and creating instances. Combined with the purpose of ontology development in this research, based on the seven-step process, a five-step method suitable for the ontology development of this research has been designed: defining the domain and purpose -> capturing domain knowledge sources -> defining core concepts and hierarchies -> defining class properties -> creating instances.

4.1. Defining the Domain and Purpose

The building object ontology to be developed in this research is explicitly related to the field of building engineering, and the purpose of the application is to represent building objects and their associated information in a structured and easy-to-query way. As shown in Figure 2, the scope of knowledge of the ontology can be limited in the following ways:

- Building Objects: Building objects refer to the various entities or concepts in the field of architecture that people pay attention to, study, design, and construct. The knowledge provisions for inspection work have a wide range of sources and cover numerous relevant elements within the architectural field, such as architectural spaces like kitchens and bathrooms, architectural components like walls and doors, and electromechanical products like elevators and plumbing fixtures.
- Object Properties: In inspection work, the properties of the objects are common inspection indicators. The property information associated with the building objects is very rich, such as basic information like geometric dimensions, spatial location information like spatial coordinates, as well as accessory properties information like safety exits.
- Object relationships: Inspection work is implemented through querying. Therefore, when querying relationships, the relationships between objects serve as an important basis for assessment. For instance, windows and doors are attached to walls, while bathroom fixtures are contained within bathrooms.

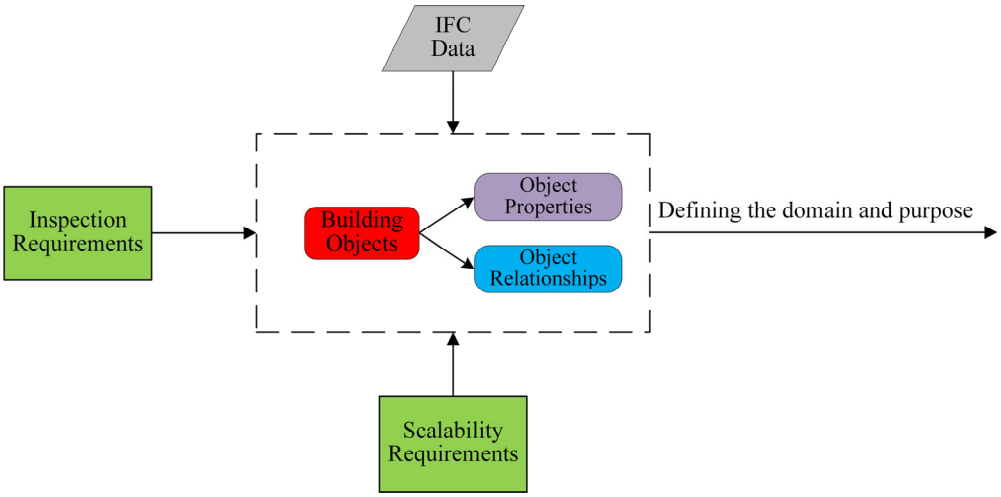

Figure 2. Defining the domain and purpose.

4.2. Capturing Domain Knowledge Sources

In order to ensure the comprehensiveness and systematicity of the core concepts contained in the ontology model, two highly relevant standards were selected: Standard for Classification and Coding of Building Information Model (GB/T 51269-2017) [44] and Classifying and Coding of Construction Product (JG/T 151-2015) [45].

The GB/T 51269-2017 [44] specifies the classification and coding standards for the information contained in the BIM model. The structural classification of the main objects involved is shown in Figure 3.

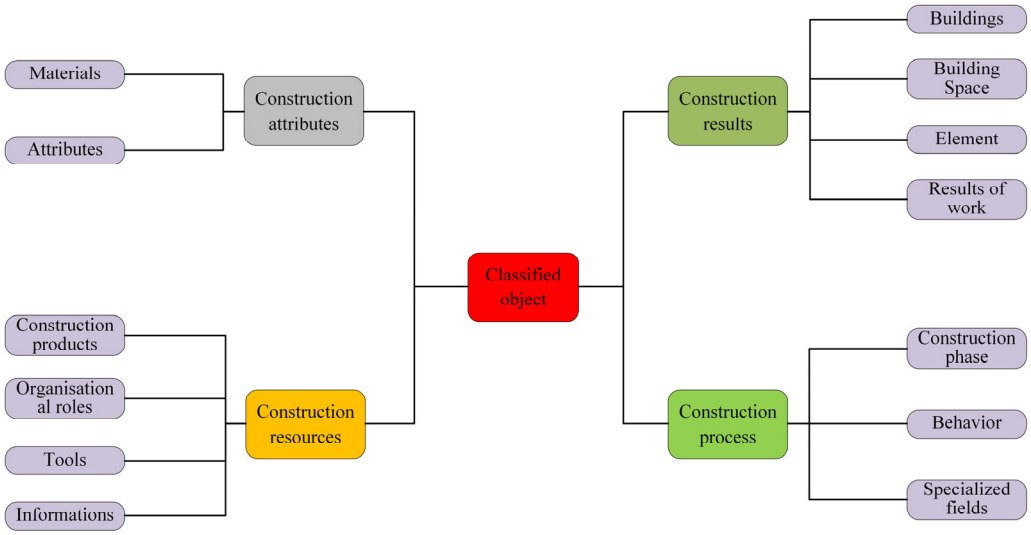

Figure 3. BIM information classification structure.

The JG/T 151-2015 [45] clearly defines construction products as products that are permanently integrated into the building entity throughout the construction and utilization processes of a building project, encompassing various materials, equipment, and combinations thereof. Based on fundamental characteristics such as materials and functions, construction products can be classified into four categories: architectural, structural, electromechanical, and civil air defense, as illustrated in Figure 4.

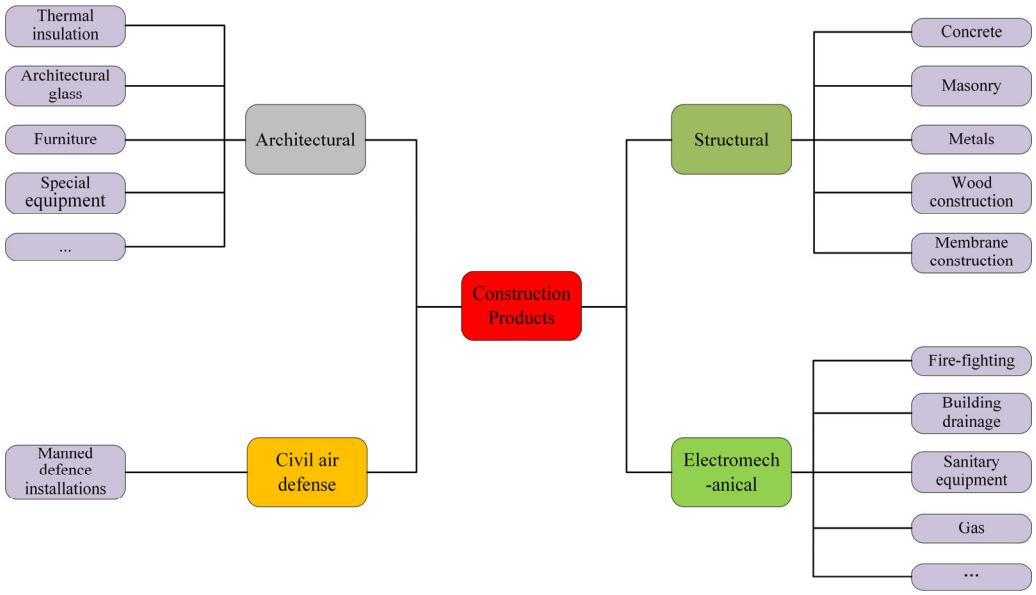

Figure 4. Construction products' specialized classification information.

The above-mentioned two standards, serving as sources of knowledge, predominantly exhibit two aspects of functionality. Firstly, they provide diversified classification methodologies for building objects, establishing a referential foundation for defining core classes and subclasses at multiple hierarchical levels within the ontology model; Secondly, they

systematically encode domain-specific terminology, supplying standardized vocabularies for class-attribute relationships across all ontological hierarchies.

The five-step method proposed in this study is highly adaptable and not restricted to the two Chinese national standards used here. It can also be effectively applied to standards from other countries. For instance, when using the U.S. standard OmniClass, the knowledge source acquisition process can focus on its classification table related to architectural elements and spatial functions, extracting hierarchical relationships and terminology to define core concepts and structures.

4.3. Defining Core Concepts and Hierarchies

Building upon the previous two sections, building objects can be systematically categorized into three primary classes: buildings, building spaces, and building elements. Buildings and building spaces undergo further subclassification based on functional characteristics and morphological attributes. The category of building elements is particularly comprehensive, encompassing diverse components including building components and building products. Figure 5 presents a detailed hierarchical classification of building objects, providing a clear visual summary of the proposed categorization framework.

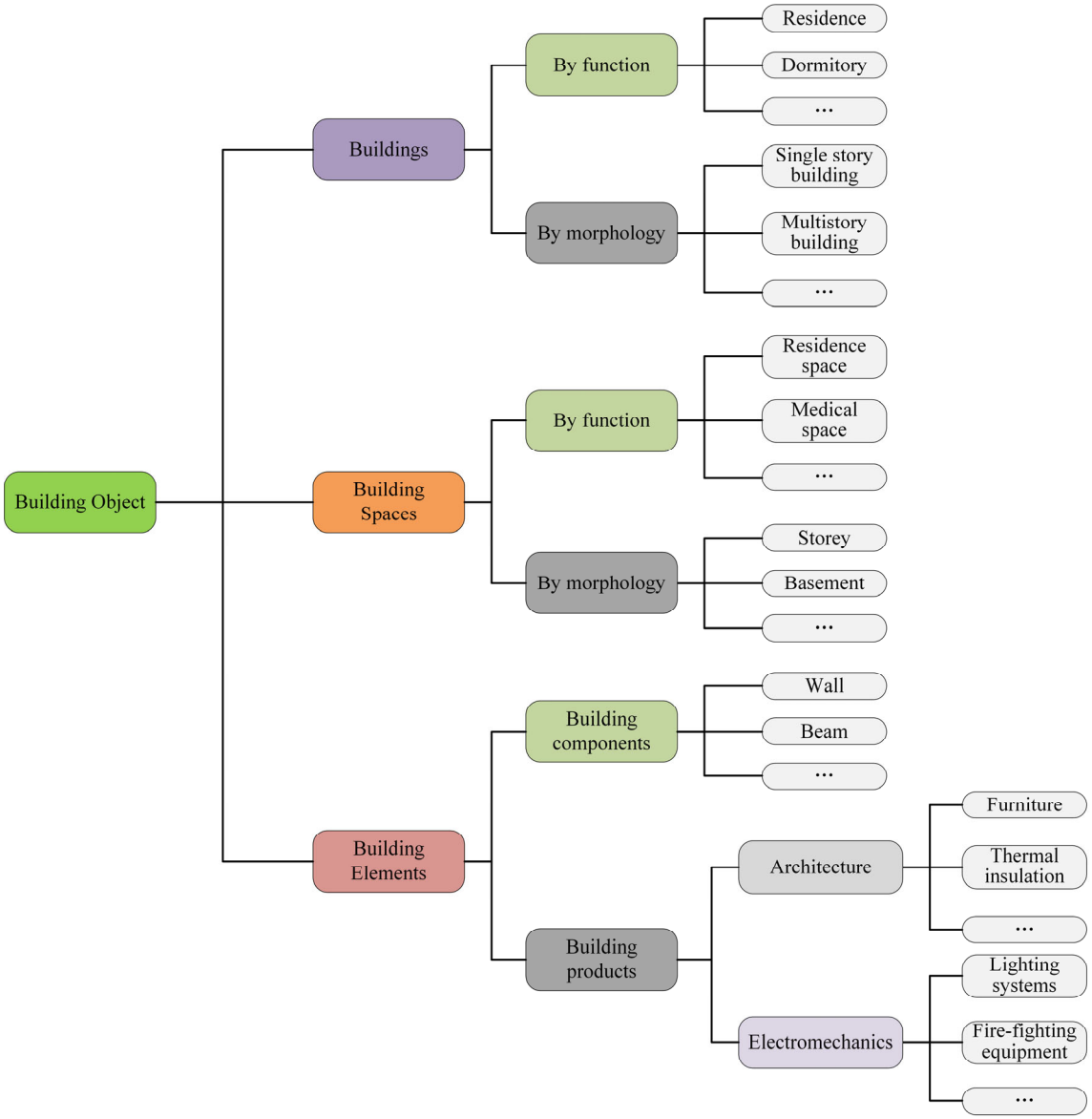

Figure 5. Building object ontology classes and layers.

Following the design of ontological classes and hierarchical structures, the subsequent step involves implementing them within an ontology editor. It is crucial to note that the ontology instantiation depends on information extracted from IFC-format BIM models. Consequently, during the instantiation mapping phase, the mapping rules not only involve the subclass concepts and attributes of building object classes within the ontology model but also depend on the class and attribute relationships contained in IfcOWL. Thus, in the ontology construction process, the building object ontology is constructed as a peer ontology to IfcOWL, which facilitates subsequent interactive mapping. A partial view of the constructed ontology is illustrated in Figure 6. Every arrow in the diagram indicates an inheritance relationship between two classes, where the class at the tail of the arrow is the superclass of the class at the arrow's head.

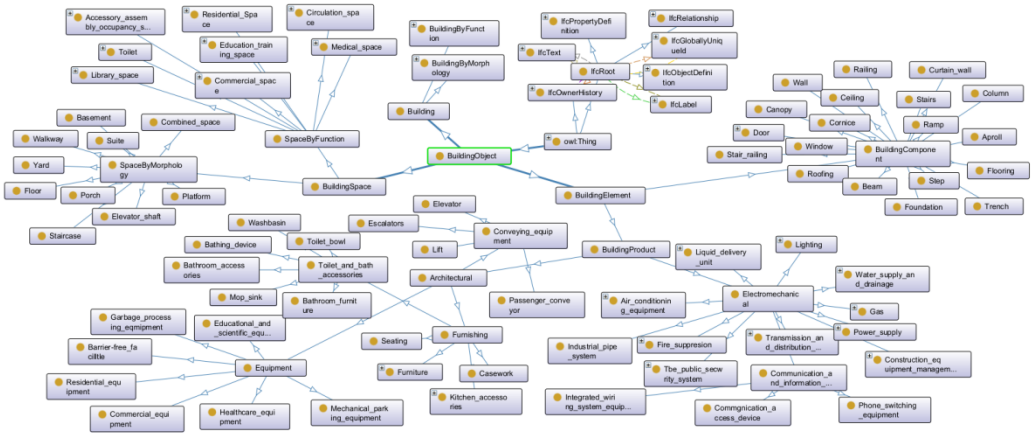

Figure 6. The structure of building ontology in Protégé.

4.4. Defining Class Properties

The properties of the ontology refer to the relationships between its constituent concepts, and such relationships serve to establish connections between two specific nodes. From the perspective of the node types, the relationships in the ontology model are typically categorized as object properties and data properties. Object properties primarily describe relationships between classes or between classes and their instances. That is, both connected nodes represent either classes or class instances. Data properties, on the other hand, establish connections between an instance and its property value. In such cases, one node is an instance of a class, while the other is the value of a specific property of that instance.

From a functional perspective, object properties can be classified as hierarchical or non-hierarchical. Hierarchical properties are mainly used to define the hierarchical relationships between classes and instances, as well as among these classes and instances themselves. In Protégé, these relationships require no manual intervention. Conversely, non-hierarchical properties must be explicitly defined during ontology modeling. These properties are generally developed to fulfill practical application requirements based on specific ontology use cases. The ontology proposed in this study is designed to express code elements, with its content derived from relationship descriptions of building objects across various codes. Table 2 lists the representative non-hierarchical object properties developed according to the purpose of this study.

Table 2. Non-hierarchical object properties.

Relationship Type	Representative Property	Description
Building to architectural space	Has	Denotes that a building contains one or multiple floors.
Architectural space to component	On	Indicates that a component is located on the corresponding floor space.
Component to component	AttachedTo	Represents an attachment relationship between components.
Architectural space to architectural space	AdjacentTo	Indicates adjacency between two architectural spaces.

Similarly, data properties also require additional definition in the code provisions; there are a large number of constraints on the property values of building objects. For example, it includes the spatial properties of construction projects, such as the area of a bedroom and the width of a corridor, and also the properties of architectural elements, like the height of a railing and the clear width of a staircase. These properties generally have direct corresponding values in the BIM model. We need to extract this information from the IFC files with more complex structural data and use our defined data properties as the relationships in the triples to connect the property values of the objects. In addition, some data properties are specifically designed to assist in the inspection and judgment, for instance, the coordinate properties representing the building objects in the global coordinate system, where "MinX", "MaxX", and "CX", respectively, represent the minimum coordinate, the maximum coordinate, and the central coordinate of the building objects in the *X*-axis direction. These property values play a significant role in determining the spatial position relationships and distances between building objects. Moreover, although the elevation property "Elevation" of the building floors is not directly used as the object of inspection, it can serve as a reference basis for judging the height of the floors.

After the development of the main object properties and data properties is completed, the construction of the ontology is basically finished. Similar to the definition of classes, the definition of relationships is parallel to the relationships in IfcOWL, jointly serving as the basis for subsequent reasoning and querying. Moreover, these relationships can be further expanded based on the ontology's application requirements.

4.5. Creating Instances

Ontology instantiation in prior studies predominantly relied on manual addition based on Protégé or semi-automated instance generation via Protégé's Cellfie plugin using tabular data. However, this paper's ontology instantiation method automatically adds instances from IfcOWL-represented building data via mapping rules. The IfcOWL can be directly generated through the IFC-to-RDF conversion tool, and the mapping rules will be elaborated in detail in the next section.

5. Rule Development

5.1. Mapping Rules Development

Mapping rules serve as the bridge for establishing the corresponding relationship between codes and building model data. The core of designing mapping rules lies in the entity expression methods between the two, that is, establishing correspondence between entity information in the IFC standard and building objects. In the IFC standard, the representation of entity information can be categorized into three types: the corresponding expression of classes, the information expression between entity objects, and the expression of the mutual relationships between entities.

In previous studies, SWRL was mostly chosen as the rule language. However, in this study, due to the addition of more complex spatial location information, the number of instances in IfcOWL has increased significantly. Using the conventional SWRL language to write mapping rules cannot achieve the expected results, which are mainly reflected in the following aspects.

Firstly, processing spatial location information (such as coordinate transformation, bounding box calculation, and entity distance) requires a large number of mathematical operations. Compared with SWRL, which cannot perform operations like coordinate transformation and vector calculation, Jena's ARQ function library can effectively support these operations. Secondly, the large-scale triples generated by IFC conversion will significantly increase the data reading time of the SWRL reasoner, thereby prolonging the reasoning time. In contrast, Jena's TDB storage engine supports efficient disk-level reading and writing, which can accelerate data reading and shorten the reasoning time. Finally, entity attributes in IFC often need to be associated with other object descriptions through long paths, and this expression is not friendly enough to SWRL; however, Jena can flexibly handle such long-path queries using the ARQ function library.

Therefore, the mapping rules of this study adopt the Jena language. In general, each Jena rule includes the body term (premise), the head term (conclusion), and the reasoning direction. Both the body term and the head term adopt the description mode of triples. The reasoning direction includes forward reasoning, backward reasoning, and bidirectional reasoning, and this study adopts forward reasoning. Based on the classification of IFC entity information expressions, the mapping rules are categorized into three types: the corresponding mapping rules of entities, the mapping rules of entity object information, and the mapping rules of relationships between entities.

5.1.1. Corresponding Mapping Rules of Entities

The mapping of entities refers to the correspondence between classes in building objects and IFC entities, which can be divided into two situations, as shown in Table 3. The first is the term mapping, the most fundamental type, where classes directly correspond to IFC entities. For example, Wall maps to IfcWall and Door to IfcDoor, corresponding to Rule 1-1 and 1-2, respectively. The second category requires differentiation using entity attribute information. For instance, spatial entities like kitchens and bedrooms all classify as IfcSpace. Here, unique attribute information (e.g., room type) must be used for differentiation, as shown in Rule 1-3 and 1-4.

Table 3. Examples of entity correspondence mapping.

Rule NO.	Example
1-1	(?x rdf:type ifc:IfcWall) -> (?x rdf:type :Wall)
1-2	(?x rdf:type ifc:IfcDoor) -> (?x rdf:type :Door)
1-3	(?x rdf:type ifc:IfcSpace) (?x ifc:longName_IfcSpatialElement ?y) (?y expr:hasString ?n) equal (?n "Bathroom") -> (?x rdf:type :Bathroom)
1-4	(?b ifc:relatedObjects_IfcRelDefinesByType ?x) (?b ifc:relatingType_IfcRelDefinesByType ?y) (?y ifc:hasPropertySets_IfcTypeObject ?p) (?p ifc:hasProperties_IfcPropertySet ?s) (?s ifc:name_IfcProperty ?i) (?i expr:hasString ?n) equal (?n "Family Name") (?s ifc:nominalValue_IfcPropertySingleValue ?v) (?v expr:hasString ?t) equal (?t "Toilet closet") -> (?x rdf:type :Toilet)

The aforementioned Rule 1-3 and Rule 1-4, respectively, correspond to the basic characteristic information and type attribute information of entities. This mapping simplification is derived from a comprehensive analysis of IFC entity information expression paths, as visualized in Figure 7. The conclusion of the rule is represented by a dashed arrow and the triples are indicated by the two ends it connects.

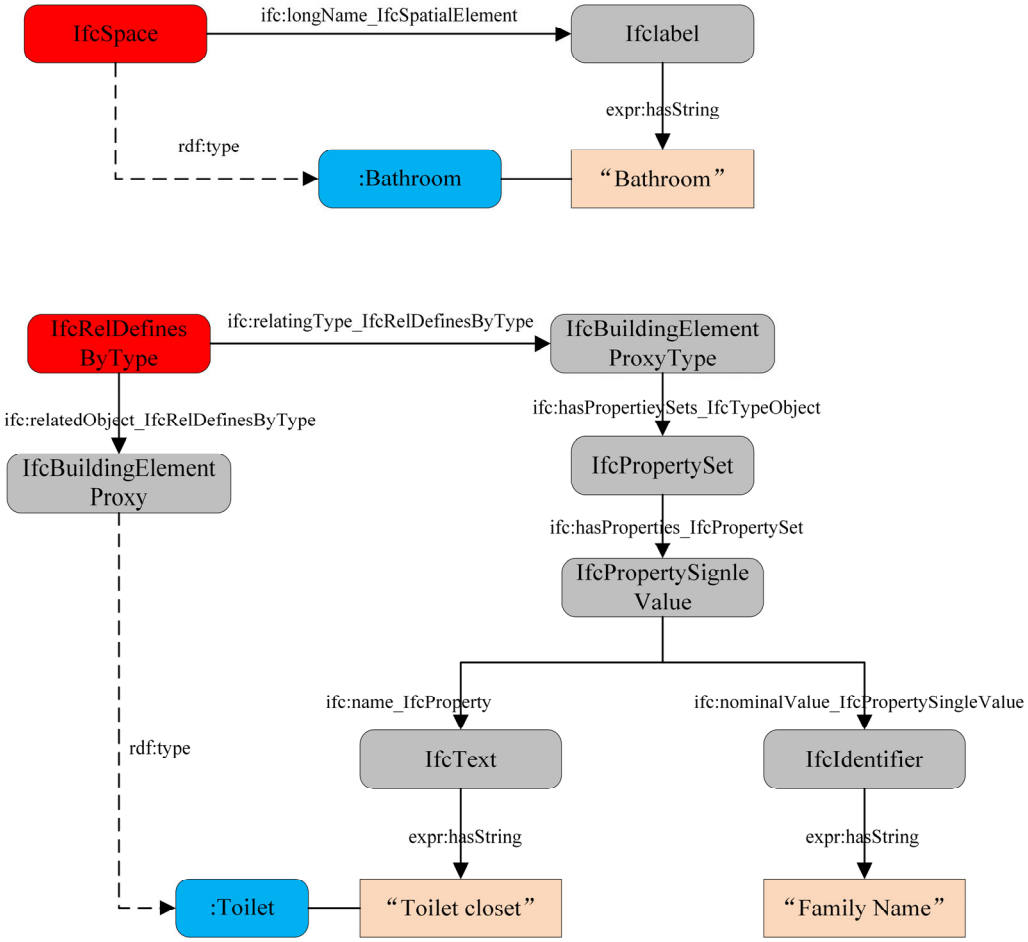

Figure 7. IFC entity information expression path.

5.1.2. Mapping Rules of Entity Object Information

In the IFC system, entity information representation methods are primarily classified into two categories: entity attribute information and entity position information, as detailed in Table 4. The first category is the mapping of entity attribute information. While similar to methods for obtaining attribute values, its core purpose is not to compare or identify entity types but to assign properties to entities. By leveraging the data properties within the ontology, the connection between entities and their attribute information is established, for example, the area attribute of spatial entities described in Rule 2-1 and the exclusive elevation attribute of floors mentioned in Rule 2-2. The second category is the mapping of entity position information, which is the most complex. When describing the spatial position of an entity within the IFC system, it is necessary to associate with numerous entities of different types, which undoubtedly increases the complexity of the mapping process. This can be specifically referred to in Rule 2-3 and Rule 2-4.

Table 4. Examples of entity object information mapping.

Rule No.	Section No.	Example
2-1		(?d ifc:relatedObjects_IfcRelDefinesByPrpperties ?x) (?x rdf:type ifc:IfcSpace) (?d ifc:relatingPropertyDefinition_IfcRelDefinesByProperties ?p) (?p ifc:hasProperties_IfcPropertySet ?s) (?s ifc:name_IfcProperty ?i) (?i expr:hasString ?a) equal (?a "Area") (?s ifc:nominalValue_IfcPropertySingleValue ?v) -> (?x :Area ?h)
2-2		(?x rdf:type :Floor) (?x ifc:elevation_IfcBuildingStorey ?s) (?s expr:hasDouble ?v) -> (?x :Elevation ?v)
2-3	1	(?w rdf:type ifc:IfcWall) (?w ifc:objectPlacement_IfcProduct ?p) (?p ifc:relativePlacement_IfcLocalPlacement ?l) noValue(?l ifc:refDirection_IfcAxis2Placement3D ?t)
	2	(?l ifc:location_IfcPlacement ?lo) (?lo ifc:coordinates_IfcCartesianPoint ?o) (?o list:hasContents ?m) (?m expr:hasDouble ?ox) (?w ifc:representation_IfcProduct ?s) (?s ifc:representations_IfcProductRepresentation ?r)
	3	(?r list:hasNext ?n) (?n list:hasNext ?k) (?k list:hasContents ?e) (?e ifc:items_IfcRepresentation ?f) (?f ifc:corner_IfcBoundingBox ?b) (?b ifc:coordinates_IfcCartesianPoint ?d) (?d list:hasContents ?j) (?j expr:hasDouble?cx) sum(?ox ?cx ?sx) -> (?w :MinX ?sx)
2-4	1	(?w rdf:type ifc:lfcWall) (?w ifc:objectPlacement_IfcProduct ?p) (?p ifc:relativePlacement_IfcLocalPlacement? l) (?l ifc:refDirection_IfcAxis2Placement3D ?t) (?t ifc:directionRatios_IfcDirection ?dr) (?dr list:hasContents? rx) (?rx expr:hasDouble ?cx) equal(?cx 0) (?dr list:hasContents ?ry) (?ry expr:hasDouble ?cy) equal(?cy 1)
	2	(?l ifc:location_IfcPlacement ?lo) (?lo ifc:coordinates_IfcCartesianPoint ?o) (?o list:hasContents ?m) (?m expr:hasDouble ?ox) (?w ifc:representation_IfcProduct ?s) (?s ifc:representations_IfcProductRepresentation ?r)
	3	(?r list:hasNext ?n)(?n list:hasNext ?k) (?k list:hasContents ?e) (?e ifc:items_IfcRepresentation ?f)(?f ifc:corner_IfcBoundingBox ?b) (?b ifc:coordinates_IfeCartesianPoint ?d)(?d list:hasNext ?j) (?j list:hasContents ?cx) difference (?ox ?cx ?lx) -> (?ele :MaxX ?lx)

In Rule 2-3, Section 1 determines whether the relative coordinate system has changed. Since the relative coordinate system remains unchanged in this scenario, the built-in function noValue () is applied for clarification. Section 2 obtains the *X*-axis coordinate of the local coordinate system's origin, while Section 3 acquires the *X*-axis coordinate value of the corner points of the entity's bounding box. Summing these two X-coordinates yields the entity's global X-coordinate value.

In contrast, Rule 2-4 exhibits greater complexity due to its altered relative coordinate system. Specifically, the local system's RefDirection parameter (defining *X*-axis orientation) changes from the default (1, 0, 0) to (0, 1, 0). As shown in Figure 8 (left), this coordinate transformation interchanges the *X*-axis directions between the local coordinate system $X_1O_1Y_1$ and the global coordinate system XOY. By applying the right-hand rule, the *Y*-axis orientation is subsequently determined. Projecting the entity's bounding box onto the XOY plane provides corner coordinates (a_1, b_1) in the $X_1O_1Y_1$. Through coordinate transformation analysis, the corresponding X-coordinate in the global system is derived as a - b_1, representing the maximum X-coordinate value of the bounding box. The Y-coordinate is similarly obtained.

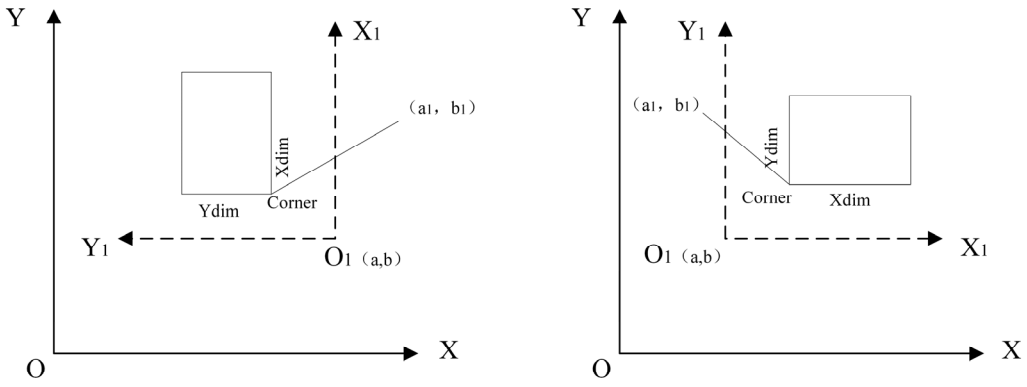

Figure 8. Demonstration of coordinate transformation.

The right side of Figure 8 depicts the default case with untransformed local coordinate axes. As a matter of fact, this transformation methodology extends to other directional configurations, including $(-1, 0, 0)$ and $(0, -1, 0)$. In such cases, the Y-axis orientation is systematically deduced based on the specified X-axis direction and the right-hand rule, followed by coordinate mapping calculations to determine equivalent global coordinate values. This framework guarantees consistent spatial transformations across diverse coordinate system configurations.

Similarly, the Y- and Z-coordinate values of the bounding box corners can be analogously derived. Furthermore, combining these derived corner coordinates with bounding box dimensions via geometric computations determines the entity's centroid coordinates, as outlined in Table 5. Specifically, Rule 2-5 calculates the bounding box's X-axis length in the global system, while Rule 2-7 utilizes the results from both Rule 2-5 and Rule 2-3 to calculate the X-coordinate of the entity's centroid. These rules systematically integrate coordinate transformations with geometric properties to ensure precise spatial parameter extraction within the global reference framework.

Table 5. Examples of the x-coordinate of the entity's centroid.

Rule No.	Example
2-5	(?w rdf:type ifc:IfcWall) (?w ifc:objectPlacement_IfcProduct ?p) (?p ifc:relativePlacement_IfcLocalPlacement ?l) noValue (?c ifc:refDirection_IfcAxis2Placement3D ?t) (?w ifc:representation_IfcProduct ?s) (?s ifc:representations_IfcProductRepresentation ?r) (?r list:hasNext ?n) (?n list:hasNext ?k) (?k list:hasContents ?e) (?e ifc:items_IfcRepresentation ?f) (?f ifc:xDim_IfcBoundingBox ?b) (?b expr:hasDouble ?xl) -> (?w :XLength ?xl)
2-6	(?w rdf:type ifc:IfcWall) (?w ifc:objectPlacement_IfcProduct ?p) (?p ifc:relativePlacement_IfcLocalPlacement ?l) (?l ifc:refDirection_IfcAxis2Placement3D ?t) (?t ifc:directionRatios_IfcDirection ?rd) (?rd list:hasContents ?rx) (?rx expr:hasDouble ?cx) equal(?cx 0) (?w ifc:representation_IfcProduct ?s) (?s ifc:representations_IfcProductRepresentation ?r) (?r list:hasNext ?n) (?n list:hasNext ?k) (?k list:hasContents ?e)(?e ifc:items_IfcRepresentation ?f) (?f ifc:yDim_IfcBoundingBox ?b) (?b expr:hasDouble ?xl) -> (?w :XLength ?xl)
2-7	(?w :MinX ?sx) (?w :XLength ?xl) quotient (?xl 2 xh) sum (?sx ?xh ?cx) -> (?w :CX ?cx)

5.1.3. Mapping Rules of the Relationship Between Entities

Semantic relationships between entities are expressed through object properties, which require extensive associations between entities and their corresponding attributes. The IFC standard categorizes inter-entity relationships into two primary types: "containment" for spatial hierarchies and "connectivity" for component interactions. A subset of these rules is presented in Table 6.

Table 6. Examples of inter-entity relationship mapping.

Rule No.	Example
3-1	(?b rdf:type ifc:IfcBuildingStorey) (?p rdf:type ifc:IfcRelContainedInSpatialStructure) (?p ifc:relatingStructure_IfcRelContainedInSpatialStructure ?b) (?p ifc:relatedElements_IfcRelContainedInSpatialStructure ?w) -> (?w :On ?b)
3-2	(?s rdf:type ifc:IfcSpace) (?p rdf:type ifc:IfcRelContainedInSpatialStructure) (?p ifc:relatingStructure_IfcRelContainedInSpatialStructure ?s) (?p ifc:relatedElements_IfcRelContainedInSpatialStructure ?w) -> (?s :SetUp ?w)
3-3	(?s rdf:type ifc:IfcSpace) (?r rdf:type ifc:IfcRelSpaceBoundary) (?r ifc:relatingSpace_IfcRelSpaceBoundary ?s) (?r ifc:relatedBuildingElement_IfcRelSpaceundary ?w) -> (?s :SorroundedBy ?w)
3-4	(?f rdf:type ifc:IfcRelFillsElement) (?f ifc:relatingOpeningElement ?o) (?v rdf:type ifc:IfcRelVoidsElement) (?v ifc:relatedOpeningElement_IfcRelVoidsElement ?o) (?f ifc:ralateBuildingElement_IfcRelFillsElement ?d) (?v ifc:relatingBuildingElement_IfcRelVoidsElement ?w) -> (?d :AttachedTo ?w)
3-5	(?s :SurroundedBy ?w) (?i :SurroundedBy ?w) (?w rdf:type wall) notEqual(?s ?i) -> (?s :AdjacentTo ?i)

Rule 3-1 and Rule 3-2 represent "containment" relationships but apply distinct semantic relations, "On" and "SetUp", respectively, to accommodate differences in entity types. Similarly, Rule 3-3 and Rule 3-4 (categorized as "connectivity" relationships) utilize differentiated semantics ("SurroundedBy" and "AttachedTo") for entity interaction modes. For these foundational rules, additional inter-entity relationships can be logically derived by integrating entity-specific attributes and pre-established semantic associations. For example, Rule 3-5 leverages the "SurroundedBy" relationship inferred from Rule 3-3 to further deduce a higher-order spatial relationship, "AdjacentTo".

5.2. Query Rules Development

5.2.1. Analysis of Code Provisions

Query rules are developed based on SPARQL. Generally, a complete specification provision consists of three parts: the constrained object, the constrained content, and the preconditions. Among them, the constrained object serves as the subject of the query. Both the constrained content and the preconditions are attached with category labels according to their contents, such as property conditions, construction conditions, spatial conditions, and basic requirements. Basic requirements are usually described without conditions. Therefore, apart from the constrained object, the other two parts are composed of content described by three types of conditions, namely property, construction, and spatial conditions, and these three types can be analyzed in a relatively quantifiable manner.

Property descriptions are typically defined using a concise structure of "property name + constraint requirement". Taking the building code provision "The usable area of a bedroom shall not be less than $9m^2$" as an example, the logical structure can be

deconstructed into three components: the constrained object (bedroom), property name (usable area), and constraint requirement (shall not be less than 9 m^2).

Construction descriptions primarily express containment relationships in spatial configurations. Such expressions can be simplified as "containment relationship + contained elements". Taking the code provision "When a bathroom is equipped with a toilet and a washbasin, its usable area shall not be less than 1.80 m^2" as an example, the semantic framework of the construction description therein can be decomposed into three components: the constrained object (bathroom), the containment relationship (equipped with), and the contained elements (toilet and washbasin).

Spatial descriptions can be systematically categorized into positional relationships and distance relationships. Distance relationships can be simplified into a structure of "associated objects + distance requirements". For instance, the clause "When the distance from the entrance door of any residential unit to a safety exit exceeds 15 m, at least two safety exits shall be provided on that floor" can be decomposed into the following semantic components: the constrained object (entrance door), the associated object (safety exit), and the threshold criterion (when the distance exceeds 15 m). Among them, positional relationships can be simplified into a structure of "spatial relationship + associated objects". For instance, the provision "The clear width of the hallway providing access to bedrooms and living rooms shall not be less than 1.00 m" can be structurally decomposed into: constrained object (hallway), spatial relationship (providing access to), and associated objects (bedrooms and living rooms). Distance relationships can be simplified into a structure of "associated objects + distance requirements". For example, the provision "When the distance from any residential unit's entrance door to a safety exit exceeds 15m, at least two safety exits shall be provided on that floor" can be decomposed into the following semantic components: constrained object (entrance door), associated object (safety exit), and distance requirements (when the distance exceeds 15 m).

Table 7 systematically categorizes three types of machine-translatable regulatory descriptions along with their semantic element compositions. It is noteworthy that within the core element expression framework, except for modal verbs, comparative operators, and specific numerical parameters, all other semantic units can be formally represented as classes or properties within a domain ontology.

Table 7. Quantifiable descriptive categories and their elemental expressions.

Description Type	Form of Expression of Core Elements	Expression of Elements
Property	property name + constraint requirements	net height, usable area, net width, ...
Construction	containment relationship + contained element	set, configure, arrange, ...
Spatial	spatial relationship + associated objects	access, adjoin, ...
	associated objects + distance requirements	be higher than, be separated by, ...

5.2.2. Natural Language to SPARQL Translation

For the translation of specification provisions, regardless of the type of description, the described content is oriented toward building objects, and what ultimately needs to be filtered out are the constrained objects that do not meet the requirements. Based on the classification of structural expressions of different specification provisions in Section 5.2.1, we have, respectively, designed query rule templates for translating different types of specification provisions into SPARQL, as shown in Tables 8 and 9. In the tables, the content enclosed in "[]" represents variables, which correspond to the core elements summarized in Table 7.

Table 8. SPARQL templates for property descriptions and construct descriptions.

Type	Template

Property description	SELECT ?s
	WHERE {
	?s rdf:type [query object].
	?s [property name] ?p.
	Filter (?p Comparison symbols [value])
	}

Construction description	SELECT ?s
	WHERE{
	?s rdf:type [query object].
	. . .
	?o rdf:type [contained element].
	?s [containment relationship] ?o.
	. . .
	}

Table 9. SPARQL templates for spatial description.

Type	Template

Positional relationship	SELECT ?s
	WHERE {
	?s rdf:type [query object].
	. . .
	?o rdf:type [associated object].
	?s [spatial relation] ?o.
	. . .
	}

Distance relationship	SELECT ?s
	WHERE {
	?s rdf:type [query object].
	. . .
	?o rdf:type [associated object];
	:Cx ?x;
	:Cy ?y;
	:Cz ?z.
	. . .
	Bind (calculation result As ?d)
	Filter (?d Comparison symbols [value])
	}

The conversion templates categorized in the table serve as fundamental building blocks, where most engineering specifications require cross-template combinatorial applications to achieve SPARQL statement conversion. Typical examples are illustrated as follows. Figure 9 illustrates an example compliance query for hallways, integrating the property description (net width should not be less than 1000 mm) and the positional relationship in the spatial description (aisle to bedroom, living room). It is worth noting that here the net width attribute of the hallway is replaced by the width parameter of the bounding box, thus also covering the relationship in the spatial description. Figure 10 illustrates an example compliance query for floors. Since this SPARQL statement is too long, a nested query is used for logical clarity. The SPARQL statement 1 is used to filter out residential units (spatial descriptions) that comply with the emergency exit distance requirements. The SPARQL statement 2 is used to filter out residential units (constructional descriptions)

up to the tenth floor. SPARQL statement 2 is nested in SPARQL statement 1 and co-nested in the main SPARQL statement.

```
SELECT ?h
WHERE{
?h  rdf:type          :Hallway; #Constraint object
        :Connected  :Bedroom;
        :Connected  :Living_room; #Positional relationships
        :XLength     ?xl;
        :YLength     ?yl.
Bind(afn:min(?xl,?yl) as ?w) #Distance relationships
Filter(?w < 1000) #Property description
}
```

Figure 9. SPARQL query rule conversion example 1.

```
SELECT  ?f  (count (?e) as ?c1)
WHERE{
?r  rdf:type  :Residential_Building.
?f  rdf:type  :Floor. #Constraint object
?d  rdf:type  :Entrance_door.
?e  rdf:type  :Emergency_exit.
  {SELECT Distinct ?r
   WHERE{
   ?r  :Has  ?f.
   ?d :Cx  ?x1;
       :Cy  ?y1; #Distance relationships
       :On  ?f. #Positional relationships
   ?e  :Cx  ?x2;
       :Cy  ?y2; #Distance relationships
       :On  ?f. #Positional relationships
  Bind(afn:sqrt ( ( (?x1-?x2) * (?x1-?x2) + (?y1-?y2) * (?y1-   ?y2) ) as ?s)
  Filter(?s > 15000) #Distance relationships
   {SELECT ?r (count (?f) as ?c2)
    WHERE{
    ?r :Has ?f
    }
    Group by ?r
    Having(count (c2) < 10) #Constructive descriptions
   } #SPARQL statement 2 for Constructive descriptions
   }
  } #SPATRQL statement 1 for Spatial descriptions
Group by ?f
Having(count (c1) < 2)
}
```

Figure 10. SPARQL query rule conversion example 2.

6. Case Study

6.1. Prototype System

To validate the feasibility of the proposed methodology, we designed a prototype verification system integrating four modules (as shown in Figure 11), encompassing core functional components: building data management, ontology development, rule development, and intelligent query. In the building data management module, we chose a 21-story residential unit as a case to verify the method. This unit has a uniform structural configuration from the 5th to the 21st floor (the standard floor model as shown in Figure 12). The IFC format file of the model was obtained through the BIM data export function, and the IFC-to-RDF conversion tool was called via the command-line tool to obtain the IfcOWL ontology. The obtained IFCOWL data was stored in the TDB database. In the ontology development module, based on the two selected Chinese standards, the five-step method proposed in this study is used to complete the building object ontology construction in Protégé. The rule development module consists of two parts: mapping rules and query rules. Specifically, the semantic mapping rules are designed based on the entity expression mechanism of IFC standards and loaded by Jena rule engine to automatically realize the semantic alignment between the building object ontology and IfcOWL ontology, thereby forming the merged ontology; secondly, through the structured analysis of building code provisions, we design SPARQL statement templates for different specification types as query rules. In the intelligent query module, we set up the TDB dataset, the merged ontology, the mapping rules, and the inference engine in the Fuseki configuration. Then, we start the server, use its web interface to run predefined SPARQL queries, and view the results in JSON or table format.

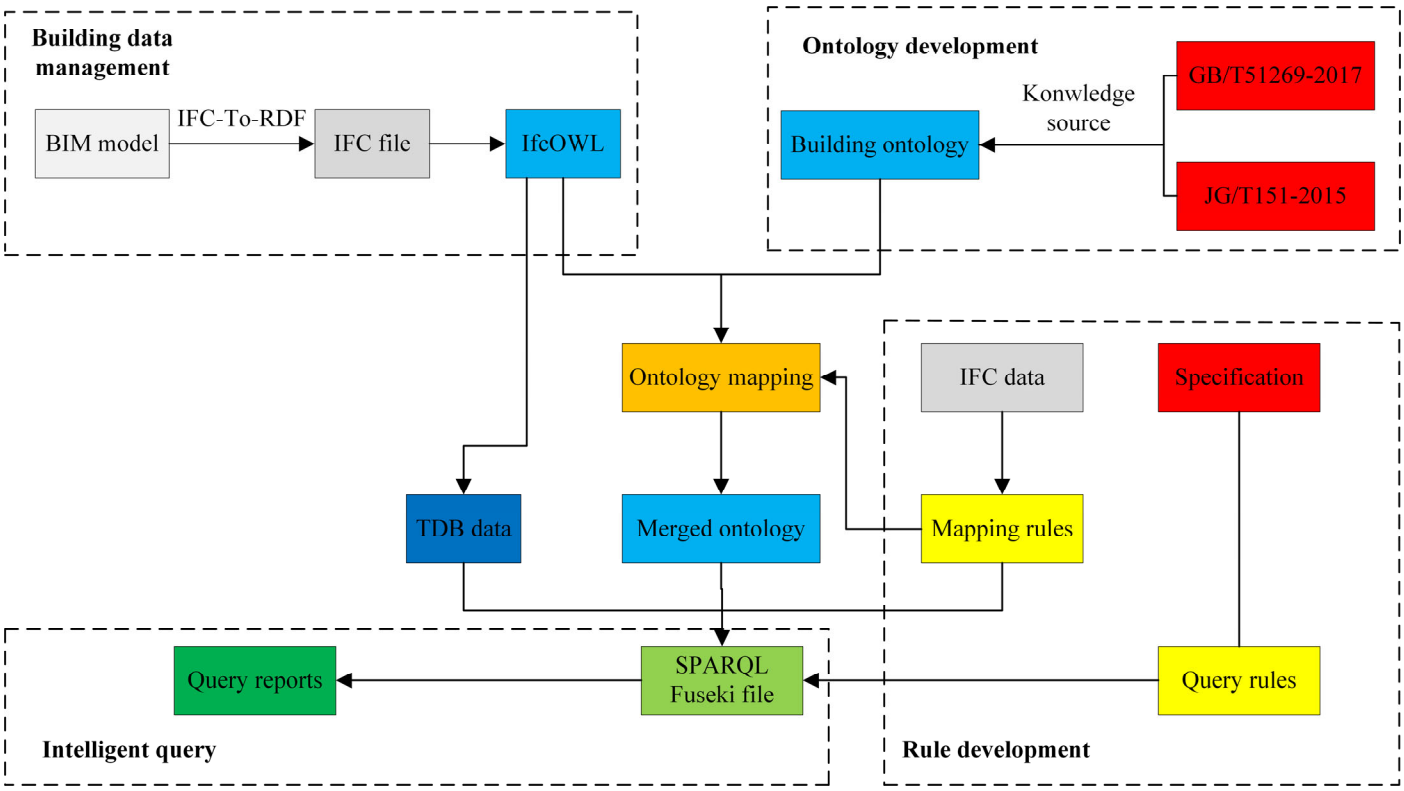

Figure 11. Structure of prototype system.

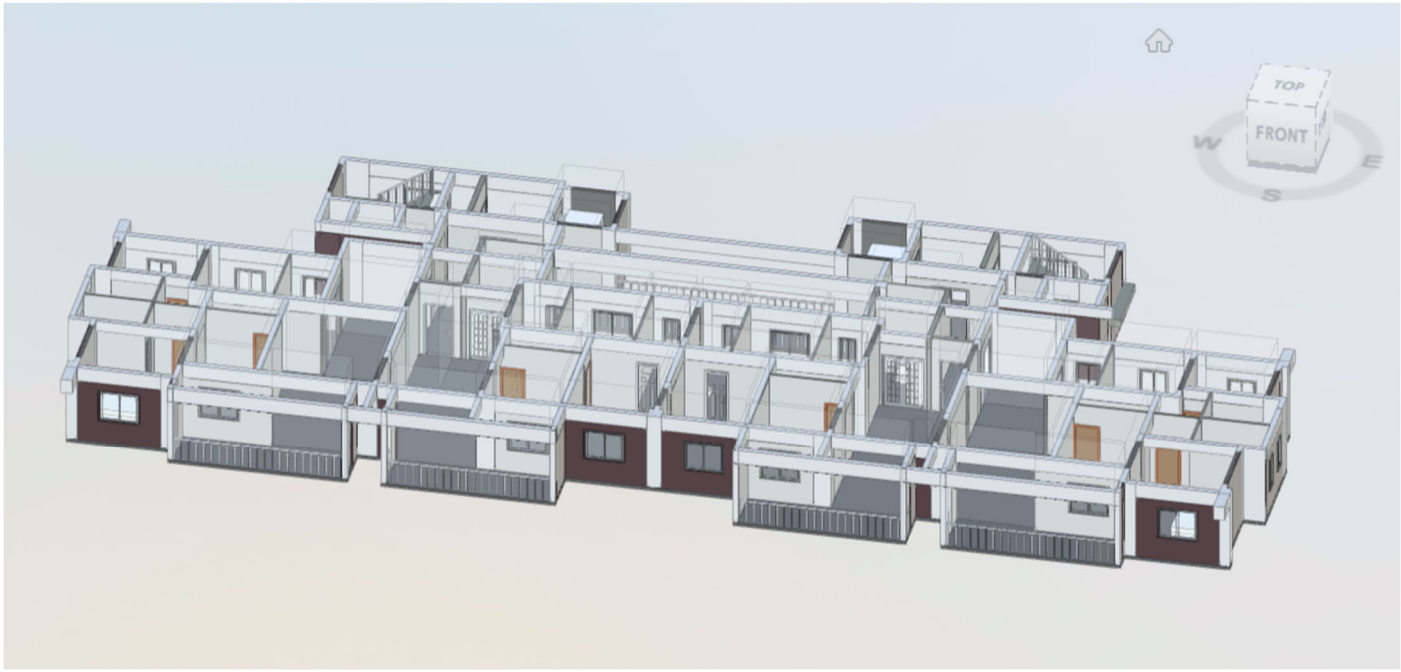

Figure 12. Standard layer modeling diagram.

The system is implemented based on Apache Jena, which provides several types of APIs, including the Ontology API for reading and manipulating ontology files, the Inference API for ontology mapping and rule-based reasoning, and the Fuseki REST API for providing SPARQL queries over HTTP.

6.2. Results of Implementation

The code compliance checking was conducted through the web-based interface of the Fuseki server, with civil residential buildings as the research subject and the Design Code for Residential Building (GB-50096) [46] serving as the compliance standard. Code provisions related to architectural properties, constructional configurations, and spatial relationships were systematically extracted from the code. Leveraging the SPARQL template established in Section 5.2, these regulatory provisions were algorithmically transformed into standardized SPARQL query rule sets through a structured conversion mechanism.

During the execution of a fundamental query, the code clause stating "The clear height of balcony railings in residential buildings with seven or more stories shall not be less than 1.10 m" was selected for verification. As shown in Figure 13, after inputting the corresponding SPARQL query into the Fuseki server's SPARQL interface, the results revealed two distinct IfcRailing entities (representing railing objects) with measured clear heights of 900 mm, both falling below the required 1100 mm threshold and thus identified as non-compliant building elements. We imported the original IFC file into BIMversion, as shown in Figure 14. By filtering components of the railing category, we obtained the location and attributed information of the railing components identified as non-compliant by the query. It can be observed that the two 900 mm railings are located at the bottom-left corner of the model, which matches the query results. Through this visualization function, users can quickly obtain the positions of components identified as non-compliant and make modifications accordingly to meet the design requirements.

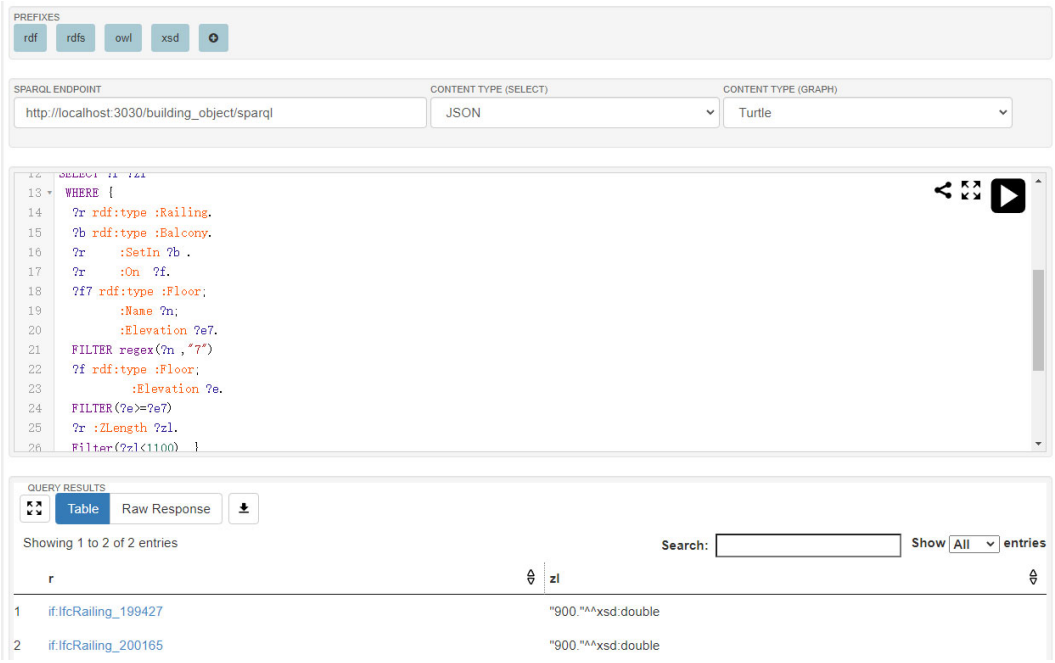

Figure 13. SPARQL query results.

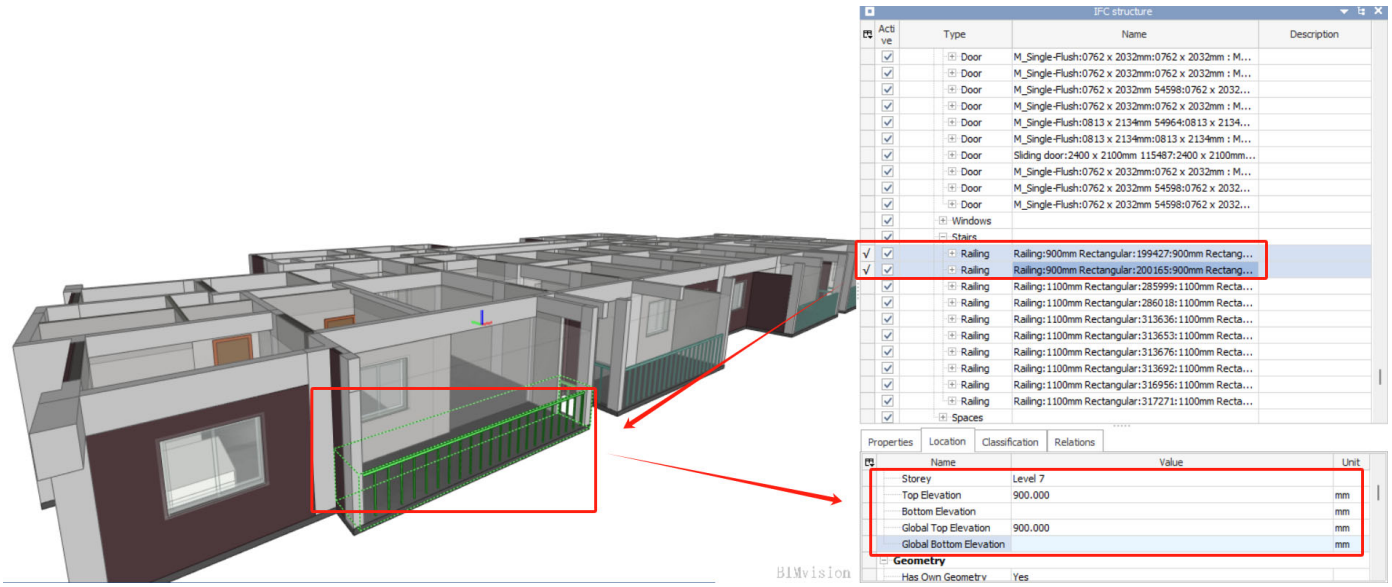

Figure 14. Visualization of query results in Bimversion.

The query process demonstrated that railing entities in the IFC framework, unlike certain family entities, lack direct spatial associations. This necessitates spatial coordinate analysis to verify containment within balcony spaces. Figure 15 illustrates the spatial positioning determination mechanism: using planar coordinates of co-located spatial and entity bounding boxes (as shown in the left panel of Figure 15), a railing is adjudged to reside within a spatial entity when its bounding box's minimum x/y coordinates exceed those of the spatial boundary while its maximum x/y coordinates remain inferior to the spatial boundary extremes. The corresponding Jena rule implementation is detailed in the right panel of Figure 15.

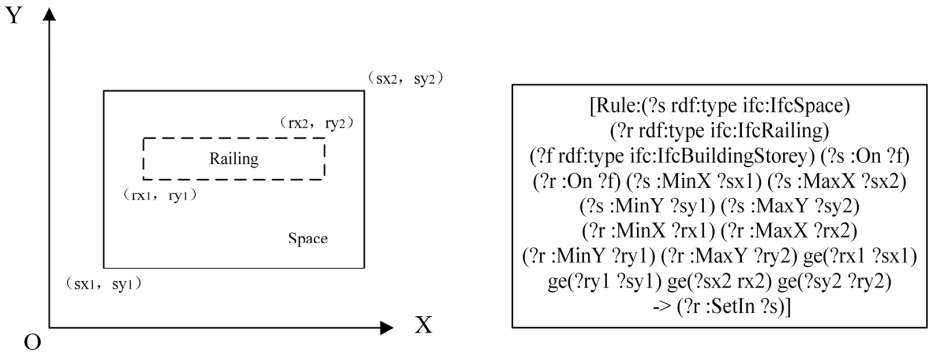

Figure 15. Rules for judging the spatial position of railing entities.

When property parameters need to be obtained through indirect data processing techniques, a typical example is illustrated by the code clause that "The window-to-floor area ratio of daylighting windows in bedrooms shall not be lower than 1/7". The query object of this clause is the bedroom, and its core parameter, the window-to-floor area ratio, cannot be directly extracted; instead, it must be indirectly derived by calculating the ratio of the spatial area to the window opening area. Similarly, the acquisition of key property indicators such as spatial clear height and clear width also requires the application of such indirect calculation methods. In practical operations, special attention should be paid to the semantic features implicit in code provisions, and the SPARQL query rules should be adaptively adjusted accordingly.

7. Conclusions

This study presents a semantic web technology-based method for code compliance checking, which significantly enhances the efficiency of building code review and effectively ensures the compliance and quality of building information models. The main research achievements are as follows:

(1) By integrating relevant standards and codes in the architectural domain, this study conducts in-depth semantic modeling of building objects and constructs a systematic building ontology model. This ontology covers diverse object types in construction engineering and features an extensible hierarchical structure, significantly improving the reusability and domain applicability of the ontology. It provides a standardized semantic framework for integrating heterogeneous building data and lays a foundation for automated compliance checking.

(2) By systematically analyzing the expression mechanisms of entity information and associated attributes in IFC standards, we designed a set of mapping rules. This rule system achieves semantic-level alignment between IfcOWL-format building model data and the building object ontology, constructing a cross-format semantic bridge.

(3) Leveraging Jena's ARQ function library, we developed a normative query rule expression system using SPARQL language. Through syntactic analysis of regulatory clauses, we established SPARQL statement templates for major constraint types, enabling combinatorial conversion of natural language specifications. This approach effectively addresses diverse query requirements for building information models.

Subsequent research can be developed in the following dimensions:

Firstly, although the current building object ontology has a complete building classification framework, it still needs to be continuously expanded and optimized when facing complex projects (e.g., hospitals, airports, etc.). Such buildings contain special functional spaces (e.g., operating rooms, departure levels of terminals), specialized equipment systems (e.g., medical gas pipelines, baggage conveyor systems) and complex spatial topological relationships, which make it difficult to comprehensively describe the attribute system and

hierarchical structure of the existing ontology. In the future, machine learning can be used to automatically analyze and identify building objects and attributes in new specifications that are not covered by the existing ontology and then expand the original ontology accordingly.

Secondly, as the model size and rule complexity grow dramatically (e.g., the number of artifacts reaches hundreds of thousands, spatial relationships and rule nesting levels deepen), the current Jena-based inference performance (e.g., memory footprint, inference speed, query response time) will be dramatically reduced. Follow-up work can explore distributed architectures (e.g., Spark-Jena integration) and rule optimization algorithms to cope with large-scale scenarios.

Finally, this study develops ontologies and rule templates based on Chinese national standards. There are significant differences in the building code systems of various countries, such as terminology, classification logic, and constraint requirements. In the future, the ontology model and rule conversion mechanism can be designed to be more flexible and configurable, allowing it to adapt more easily to the specific code requirements of different countries or regions. This will enhance the international applicability of the framework.

Author Contributions: Conceptualization, L.J. and M.C.; methodology, L.J. and M.C.; software, C.C. and Y.J.; validation, L.J., M.C., C.C. and Y.J.; formal analysis, L.J.; investigation, M.C.; resources, L.J.; data curation, M.C. and C.C.; writing—original draft preparation, L.J., M.C. and C.C.; writing—review and editing, L.J., M.C., C.C. and Y.J.; visualization, C.C.; supervision, M.C. and Y.J.; project administration, L.J.; funding acquisition, L.J. and C.C. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the Science and Technology Plan Project of Jiangxi Geological Bureau (Grant No. 2023JXDZKJKY07); the Science and Technology Plan Project of Jiangxi Geological Bureau (Grant No. 2021JXDZ70001); and the Science and Technology Plan Project of Jiangxi Coalfield Geology Bureau (Grant No. 2020JXMD70003).

Data Availability Statement: The original contributions presented in the study are included in the article, further inquiries can be directed to the corresponding author.

Acknowledgments: The authors thank all the personnel who either provided technical support or helped with data collection. We also acknowledge all the reviewers for their useful comments and suggestions.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Tan, X.; Hammad, A.; Fazio, P. Automated code compliance checking for building envelope design. *J. Comput. Civ. Eng.* **2010**, *24*, 203–211. [CrossRef]
2. Soliman-Junior, J.; Tzortzopoulos, P.; Baldauf, J.P.; Pedo, B.; Kagioglou, M.; Formoso, C.T.; Humphreys, J. Automated compliance checking in healthcare building design. *Autom. Constr.* **2021**, *129*, 103822. [CrossRef]
3. Schuk, V.; Jiménez, M.P.; Martin, U. Technical specifications to meet the requirements of an Automatic Code Compliance Checking tool and current developments in infrastructure construction. *Results Eng.* **2022**, *16*, 100650. [CrossRef]
4. Ding, Y.; Liu, M.; Luo, X. Safety compliance checking of construction behaviors using visual question answering. *Autom. Constr.* **2022**, *144*, 104580. [CrossRef]
5. Zhou, Y.; She, J.; Huang, Y.; Li, L.; Zhang, L.; Zhang, J. A Design for Safety (DFS) Semantic Framework Development Based on Natural Language Processing (NLP) for Automated Compliance Checking Using BIM: The Case of China. *Buildings* **2022**, *12*, 780. [CrossRef]
6. Sing, T.F.; Zhong, Q. Construction and real estate NETwork (CORENET). *Facilities* **2001**, *19*, 419–428. [CrossRef]
7. Nawari, N. A Generalized Adaptive Framework (GAF) for automating code compliance checking. *Buildings* **2019**, *9*, 86. [CrossRef]
8. Dimyadi, J.; Governatori, G.; Amor, R. Evaluating legaldocml and legalruleml as a standard for sharing normative information in the aec/fm domain. In Proceedings of the Joint Conference on Computing in Construction (JC3), Heraklion, Greece, 4–7 July 2017.
9. Eastman, C.; Lee, J.-M.; Jeong, Y.-S.; Lee, J.-K. Automatic rule-based checking of building designs. *Autom. Constr.* **2009**, *18*, 1011–1033. [CrossRef]

10. Pauwels, P.; Van Deursen, D.; Verstraeten, R.; De Roo, J.; De Meyer, R.; Van de Walle, R.; Van Campenhout, J. A semantic rule checking environment for building performance checking. *Autom. Constr.* **2011**, *20*, 506–518. [CrossRef]
11. Hjelseth, E.; Nisbet, N. Capturing normative constraints by use of the semantic mark-up RASE methodology. In Proceedings of the CIB W78-W102 Conference, Sophia Antipolis, France, 26–28 October 2011.
12. Goh, W.; Jang, J.; Park, S.I.; Choi, B.-H.; Lim, H.; Zi, G. Automated Compliance Checking System for Structural Design Codes in a BIM Environment. *KSCE J. Civ. Eng.* **2024**, *28*, 4175–4189. [CrossRef]
13. Zhao, J.-X.; Zhang, Z.-M.; Tian, X.-T.; Jia, X. Ontology & its applications in mechanical engineering. *Comput. Integr. Manuf. Syst.* **2007**, *13*, 727–737.
14. Neches, R.; Fikes, R.E.; Finin, T.; Gruber, T.; Patil, R.; Senator, T.; Swartout, W.R. Enabling technology for knowledge sharing. *AI Mag.* **1991**, *12*, 36.
15. Farghaly, K.; Soman, R.K.; Zhou, S.A. The evolution of ontology in AEC: A two-decade synthesis, application domains, and future directions. *J. Ind. Inf. Integr.* **2023**, *36*, 100519. [CrossRef]
16. Schmachtenberg, M.; Bizer, C.; Paulheim, H. Adoption of the linked data best practices in different topical domains. In Proceedings of the Semantic Web–ISWC 2014: 13th International Semantic Web Conference, Riva del Garda, Italy, 19–23 October 2014; Springer: Berlin/Heidelberg, Germany, 2014. Part I 13.
17. Radulovic, F.; Poveda-Villalón, M.; Vila-Suero, D.; Rodríguez-Doncel, V.; García-Castro, R.; Gómez-Pérez, A. Guidelines for Linked Data generation and publication: An example in building energy consumption. *Autom. Constr.* **2015**, *57*, 178–187. [CrossRef]
18. Demir, S.; Garrett, J.H., Jr.; Akinci, B.; Omer, A. A semantic web-based approach for representing and reasoning with vocabulary for computer based standards processing. In Proceedings of the 2010 International Conference on Computing in Civil and Building Engineering (ICCCBE'10), Nottingham, UK, 30 June–2 July 2010.
19. Bravo, M.; Reyes, L.F.H.; Reyes, J.A. Methodology for ontology design and construction. *Contad. Adm.* **2019**, *64*, 134. [CrossRef]
20. Zhong, B.T.; Ding, L.; Luo, H.; Zhou, Y.; Hu, Y.; Hu, H. Ontology-based semantic modeling of regulation constraint for automated construction quality compliance checking. *Autom. Constr.* **2012**, *28*, 58–70. [CrossRef]
21. Lu, Y.; Li, Q.; Zhou, Z.; Deng, Y. Ontology-based knowledge modeling for automated construction safety checking. *Saf. Sci.* **2015**, *79*, 11–18. [CrossRef]
22. Ma, Z.; Zhu, H.; Xiang, X.; Turk, Ž.; Klinc, R. Automatic compliance checking of BIM models against quality standards based on ontology technology. *Autom. Constr.* **2024**, *166*, 105656. [CrossRef]
23. Fitkau, I.; Hartmann, T. An ontology-based approach of automatic compliance checking for structural fire safety requirements. *Adv. Eng. Inform.* **2024**, *59*, 102314. [CrossRef]
24. Curry, E.; O'Donnell, J.; Corry, E.; Hasan, S.; Keane, M.; O'Riain, S. Linking building data in the cloud: Integrating cross-domain building data using linked data. *Adv. Eng. Inform.* **2013**, *27*, 206–219. [CrossRef]
25. Lee, Y.-C.; Eastman, C.M.; Solihin, W. Logic for ensuring the data exchange integrity of building information models. *Autom. Constr.* **2018**, *85*, 249–262. [CrossRef]
26. Zhong, B.; Gan, C.; Luo, H.; Xing, X. Ontology-based framework for building environmental monitoring and compliance checking under BIM environment. *Build. Environ.* **2018**, *141*, 127–142. [CrossRef]
27. Theiler, M.; Smarsly, K. IFC Monitor—An IFC schema extension for modeling structural health monitoring systems. *Adv. Eng. Inform.* **2018**, *37*, 54–65. [CrossRef]
28. Pauwels, P.; de Farias, T.M.; Zhang, C.; Roxin, A.; Beetz, J.; De Roo, J.; Nicolle, C. A performance benchmark over semantic rule checking approaches in construction industry. *Adv. Eng. Inform.* **2017**, *33*, 68–88. [CrossRef]
29. Smith, M.K.; Welty, C.; Mcguinness, D.L. *Owl Web Ontology Language Guide*; W3C: Cambridge, MA, USA, 2004.
30. Schevers, H.; Drogemuller, R. Converting the industry foundation classes to the web ontology language. In Proceedings of the 2005 First International Conference on Semantics, Knowledge and Grid, Washington, DC, USA, 27–29 November 2005.
31. Pauwels, P.; Terkaj, W. EXPRESS to OWL for construction industry: Towards a recommendable and usable ifcOWL ontology. *Autom. Constr.* **2016**, *63*, 100–133. [CrossRef]
32. Zhang, H.; Zhao, W.; Gu, J.; Liu, H.; Gu, M. Semantic web based rule checking of real-world scale BIM models: A pragmatic method. *Semant. Web* **2017**, *1*, 1–2.
33. Du, J.; Yu, A.; Sugumaran, V.; Hu, Y.; Yu, G.; Zhang, J. Ontological reasoning for automated tunnel defect diagnosis and root cause identification. *Autom. Constr.* **2025**, *178*, 106362. [CrossRef]
34. Wu, C.; Wu, P.; Wang, J.; Jiang, R.; Chen, M.; Wang, X. Ontological knowledge base for concrete bridge rehabilitation project management. *Autom. Constr.* **2021**, *121*, 103428. [CrossRef]
35. Soman, R.K.; Molina-Solana, M.; Whyte, J.K. Linked-Data based Constraint-Checking (LDCC) to support look-ahead planning in construction. *Autom. Constr.* **2020**, *120*, 103369. [CrossRef]
36. Zhang, Z.; Ma, L.; Broyd, T. Rule capture of automated compliance checking of building requirements: A review. *Proc. Inst. Civ. Eng. Smart Infrastruct. Constr.* **2023**, *176*, 224–238. [CrossRef]

37. Lee, J.-K.; Eastman, C.M. Demonstration of BERA language-based approach to offsite construction design analysis. In *Offsite Production and Manufacturing for Innovative Construction*; Routledge: London, UK, 2019; pp. 129–162.
38. Lee, Y.-C.; Ghannad, P.; Dimyadi, J.; Lee, J.-K.; Solihin, W.; Zhang, J. A comparative analysis of five rule-based model checking platforms. In Proceedings of the Construction Research Congress 2020, Tempe, VA, USA, 8–10 March 2020; American Society of Civil Engineers Reston: Reston, VA, USA.
39. Zhu, S.; Zhou, J.; Cheng, L.; Fu, X.; Wang, Y.; Dai, K. Research on a BIM Model Quality Compliance Checking Method Based on a Knowledge Graph. *J. Comput. Civ. Eng.* **2025**, *39*, 04024049. [CrossRef]
40. Menzel, C.P.; Mayer, R.J.; Painter, M.K. *IDEF5 Ontology Description Capture Method: Concepts and Formal Foundations*; Armstrong Laboratory: Arlington, TX, USA, 1992.
41. Gruninger, M.; Fox, M.S. The design and evaluation of ontologies for enterprise engineering. In Proceedings of the Workshop on Implemented Ontologies, European Conference on Artificial Intelligence (ECAI), Amsterdam, NL, USA, 8–12 August 1994.
42. Fernández-López, M.; Gómez-Pérez, A.; Juristo Juzgado, N. Methontology: From ontological art towards ontological engineering. In *Proceedings of the AAAI97 Spring Symposium*; Springer: Berlin/Heidelberg, Germany, 1997.
43. Uschold, M.; Gruninger, M. Ontologies: Principles, methods and applications. *Knowl. Eng. Rev.* **1996**, *11*, 93–136. [CrossRef]
44. *GB/T 51269-2017*; Standard for Classification and Coding of Building Information Model. China Architecture & Building Press: Beijing, China, 2017.
45. *JG/T 151-2015*; Classifying and Coding of Construction Products. China Architecture & Building Press: Beijing, China, 2015.
46. *GB50096-2011*; Design Code for Residential Buildings. China Architecture & Building Press: Beijing, China, 2011.